

Zero-Shot Cross-Embodiment Reuse of a Frozen Humanoid Motion Planner via Analytic Alignment and Closed-Loop-Selected Residual Heads

Sitarama Chekuri Patrick Rose (Ultimate Bots) Claude Fable 5 (Anthropic)

Draft v0.3 — 2026-08-13

Abstract

Generative motion planners for humanoids are expensive to train and are increasingly available only as pretrained checkpoints whose training distribution cannot be reproduced: the source planner in this work was trained on a proprietary 350k-clip motion collection, of which only small public subsets exist — so further training on available data would degrade the model rather than extend it. Such assets can be run, but not meaningfully retrained. We show that a 2M-step motion planner trained entirely on one humanoid (Unitree G1) can drive a morphologically similar humanoid (AgiBot X2 Ultra) at deployment quality without a single gradient step on its weights. A purely analytic wrap — per-joint affine alignment fitted from a paired retargeting corpus, plus stride, height, and intent scaling — achieves zero-shot transfer that outperforms both planners natively trained for the target robot on closed-loop stability metrics. Small residual heads (83k parameters, decoder-side only) trained on 33k paired clips close most of the remaining kinematic gap. Our central finding is methodological: offline fit and closed-loop viability diverge sharply. The best-fitting head architectures destabilized the downstream whole-body tracker through train/deploy input mismatches invisible to validation metrics, and fitting the reference height exactly (1.3 mm) removed a floating-height convention the tracker depends on. Model selection by closed-loop rollout — never by reconstruction error — plus export-time channel masking and strength scaling turned one trained model into a searchable family whose winning configuration passed every gate: recovery steps at or below the native planners’, reference smoother than the source wrap, and a 10x improvement over the deployed baseline on operator driving tapes. We further exploit the planner’s template-conditioning interface — a documented feature of its architecture — as a repair channel for specific behaviors on the new embodiment: operator-recorded gait-initiation templates, harvested by jerk metrics and routed by a stateless standstill detector, fixed a core-native first-stride surge without any training. We report the full experimental ledger, including two closed-loop-falsified architectures (and a third rejected on validation), on one closely-matched embodiment pair, and characterize the morphological-similarity preconditions the method relies on.

1 Introduction

A robot demo needed a smooth slow walk. The target robot — an AgiBot X2 Ultra driven by a generative motion planner feeding a 50 Hz RL whole-body tracking policy (SONIC, “the tracker”) — had two planners of its own, both undertrained relative to their reference architecture because GPU budget, not data, was the binding constraint: a deployed model trained early in the project on a small corpus (single 8xH100 node, roughly a quarter of the source planner’s optimization schedule), and a fresh 32-GPU cloud run on a 129k-clip corpus, wound down at a tenth of that schedule when

the fleet’s costs came due. Neither delivered the walk. The deployed planner produced acceptable but correction-heavy gait; the new planner, despite better numbers on every offline metric we measured, lost to it in closed loop where it mattered — on the seam sweeps and the primary driving tape (Section 3). Training to the source planner’s maturity — roughly 10,000 H100-hours per Section 3 — was not an option we could afford to exercise.

We also had a third planner we could not train at all. The source robot’s planner (Unitree G1) is the same architecture at identical tensor widths, trained by its authors to 2M optimization steps per model — roughly four to ten times the schedule our native chains reached — and available to us as a weights-only asset. We can retrain anything on the target robot’s side; we can never retrain the source planner. This asset asymmetry defines the setting.

The bet this paper reports: the planner is kinematic — it emits joint references, not torques — so the embodiment gap is retargeting, not dynamics, and the tracker downstream acts as a projection onto the target robot’s feasible manifold. If that is right, a frozen planner should port across morphologically close humanoids with an analytic joint-space alignment (“the wrap”) and, at most, small learned residual corrections (“the heads”).

It was right, and cheaper than expected. The wrap alone — per-joint affine alignment fitted from a paired retargeting corpus, plus stride, height, and intent scaling, fused into one graph presenting the target planner’s exact contract — beat both native planners zero-shot on the program’s headline closed-loop metric, unscheduled corrective steps (“recovery steps”): {1,2,3,4} across seeds at walk 0.35 m/s versus 6 for the deployed planner and 18–36 for the new one, and roughly 4x fewer recovery steps on recorded operator driving tapes. Adding 83k parameters of per-frame residual heads, trained in minutes-scale runs on one workstation GPU, produced a configuration that passed every gate ({1,2,1}) and an operator verdict of ready to ship for the demo walk.

The contribution we consider most transferable is not the transfer result itself but the *selection discipline that produced it*: offline fit and closed-loop viability diverge, and every promotion decision in this work had to be made against the pull of the offline metrics. The paper develops that thesis three separate times. The natively trained planner improved every offline metric and lost in closed loop. The two best-fitting head architectures destabilized the tracker through train/deploy input mismatches invisible to validation error, one of them making the system fall where no configuration had fallen before. And fitting the reference height to 1.3 mm — offline perfection — removed a floating-height convention the tracker depends on. Every promotion decision in this program was made by closed-loop rollout; every time we let an offline metric argue, it argued wrong.

Contributions: **(C1)** zero-shot planner transfer via analytic alignment, beating both native planners; **(C2)** a residual-head architecture study whose failures are reported as first-class results, with a deploy-distribution audit principle; **(C3)** closed-loop model selection with export-time masking and strength-scaling knobs, and a two-sided height ablation showing reference conventions are load-bearing; **(C4)** template-library injection — including operator-captured gait-initiation templates — as a training-free control surface of the frozen core; **(C5)** an operator-in-the-loop evaluation protocol with a published failure ledger, including two tape-parsing traps, one of which passed a quantitative gate and a human feel-test before an FK audit caught it.

We claim transfer for one closely matched embodiment pair, and we characterize the morphological preconditions the method relied on (Section 5). We do not claim “any to any.”

2 Related Work

Cross-embodiment transfer for humanoids. The closest setting to ours is Any2Any (Yang et al. 2026), which transfers G1-pretrained whole-body tracking *policies* to new humanoids via analytic kinematic alignment (scattering matrix, axis decoupling, closed-chain Jacobian) followed by LoRA-scoped RL adaptation at roughly 1% of the original compute. We lean on two of its ablations: alignment-first is load-bearing (full fine-tuning without alignment loses to LoRA with it), and input-only adaptation is unstable. Our setting differs in what is transferred: we move the *kinematic planner* and leave the target robot’s own tracker in place. Because the planner emits references rather than torques, there is no dynamics gap to close with RL — the tracker projects the reference onto the target’s feasible manifold — and the adaptation collapses to supervised regression on paired kinematics. This is why our transfer is zero-shot where Any2Any’s requires reinforcement learning.

The planner line. The frozen core is a MotionBricks-family generative planner (Wang et al. 2026): a multihead VQ-VAE motion tokenizer with root and pose prediction models, conditioned on a short context window, velocity intent, and a template clip library. Both the source and target planners in this paper are this architecture at identical widths (Section 5); the paper’s method exploits that identity rather than contributing to the architecture.

Motion retargeting. Classical and learned retargeting provides our loss vocabulary and failure taxonomy: FK-space and end-effector objectives (Aberman et al. 2020; Villegas et al. 2018), residual corrections over an analytic map with semantics-preserving regularizers (J. Zhang et al. 2023), skeleton-conditioned generalization (Lee et al. 2023), and the observation that retargeting quality gates downstream policy learning (He et al. 2024; Araújo et al. 2025). Our paired corpus is itself the product of an offline G1-to-X2 retargeting pipeline, and our corpus gates (including an IK branch-flip detector) follow this line.

Residual and adapter learning around frozen models. The heads are zero-initialized residuals over the analytic wrap, so at step zero the system is bit-exact the analytic baseline — the ControlNet initialization pattern (L. Zhang, Rao, and Agrawala 2023), also standard in residual policy learning (Silver et al. 2018). Staged fitting (affine terms before residuals) follows LP-FT (Kumar et al. 2022). Low-rank adaptation of the core itself (Hu et al. 2022) was planned as an escalation stage and never needed. Frozen-backbone reprogramming — learned input/output codecs around an untouched generative model — is the nearest published template: Time-LLM (Jin et al. 2024) for time series, PULSE-style distillation through a frozen core (Luo et al. 2024), and MotionGlot’s per-embodiment codecs (Harithas and Sridhar 2025), though MotionGlot co-trains its backbone. To our knowledge no published work does exactly what we do here: a frozen generative motion prior with trained boundary codecs fitted on paired retargeting data, selected in closed loop.

Closed-loop training and selection. Our selection discipline — promotion by rollout metrics, never by reconstruction error — is the evaluation-side analogue of closed-loop supervised training methods such as CAT-K (Z. Zhang et al. 2025), which rolls a tokenized autoregressive model on its own outputs and supervises visited states. We adopted the weaker, cheaper form (closed-loop *selection* over offline-trained candidates) and found it sufficient; closed-loop *training* of the heads remains future work. The recurring observation that open-loop motion metrics fail to rank closed-loop behavior echoes exposure-bias findings in autoregressive generation and in trajectory forecasting (Z. Zhang et al. 2025, and references therein).

Positioning. In one sentence: prior work transfers policies with RL, or retargets motions offline, or

adapts frozen backbones in other domains; we show that for kinematic motion planners feeding a competent tracker, cross-embodiment reuse needs only an analytic wrap plus tiny closed-loop-selected residual heads — and that the selection protocol, not the architecture, is the hard part.

3 The Case for Borrowing: Native Training and the Budget Wall

Before borrowing a foreign planner, we did the obvious thing: we trained our own. Twice. This section reports why that failed, because the failure mode is what motivates everything after it: not a data wall but a *compute wall*. Cost ended training early; the deficit arrived as missing optimization steps; and the damage those missing steps do is invisible to offline metrics — it surfaces only in closed loop, as recovery steps.

The target robot’s planner chain was first trained from scratch on a 33k-clip curated corpus, on a single 8xH100 node to roughly 500k steps per model (VQVAE and pose ~500k; root 315k) — a quarter of the source schedule on a quarter of the source’s GPU count — then retrained at scale: a 129,363-clip corpus (97.4% SMPL-sourced) on a 32-GPU cloud fleet (Section 4). Budget constraints ended the big run at a fraction of the source planner’s optimization schedule. The borrowed G1 chain has 2M steps on each of its three models; the native X2 chain reached 500k (VQVAE), 600k (root), and 200k (pose) before fleet wind-down.

On paper, the big run was a success. Every offline metric improved over the prior native chain: VQVAE reconstruction error fell 52%, root target-position error fell 43%, pose top-5 accuracy reached 95%. Had we promoted on those numbers — and an earlier reference-space evaluation round did argue for promotion — we would have shipped a regression. In closed loop, with the tracker in the loop and recovery steps as the metric, the big-run planner *lost to the years-old deployed planner on the seam sweeps and on operator tape A*: 18–36 recovery steps per seam-sweep run against the old planner’s 6, with sparse, low-cadence strides. (The loss is not universal — on operator tape B the big run scored 24 extra steps against the old planner’s 34 — but the demo-critical regimes all sided with the old planner.) The diagnosis is undertraining for its corpus size: 200k pose-model steps against a 129k-clip corpus leaves the model weakest exactly in the closed-loop regime — replan-seam continuity, gait rhythm under perturbation — that reconstruction metrics do not see. This is the paper’s thesis in its first appearance: offline fit and closed-loop viability diverge, and the divergence here was not a few percent but a factor-of-several loss hiding behind across-the-board offline gains.

The cost asymmetry is the headline. The source planner’s reported training budget is 32 H100 GPUs for 13 days — approximately **10,000 H100-hours** (32 x H100 x 13 days = 9,984 GPU-hours) — per its authors. Our native big-run attempt spent a 32-H100 fleet for a day-class burn (~1,500 H100-hours, exact accounting in Appendix C) and produced a chain that loses in closed loop to a years-old baseline. Pushing the native chain from where it stopped to the source’s 2M-step maturity would have required committing the remainder of that ~10,000 H100-hour schedule, with no guarantee the closed-loop crossover arrives before the budget does.

The complete adaptation reported in this paper — the analytic wrap (zero training) plus every residual-head run, ablation, and gate in the S1 study — consumed **a few hours on a single workstation RTX 5090** (~6 GPU-hours: five head-training runs at 25–40 steps/s, ~35 min each, plus evaluation rollouts). Reaching the borrowed planner’s maturity natively would have cost roughly **three orders of magnitude more GPU-time than the adaptation that matched it**. All accounting in this paper is stated in GPU-hours; no monetary figures are used.

Two qualifications keep this honest. First, the comparison is not free-floating: the adaptation only

costs ~6 GPU-hours because a fully trained source planner *exists* and because the target embodiment is morphologically close to the source (Section 5); the 10,000 H100-hours were spent by someone. The claim is about the *marginal* cost of serving a new sibling embodiment, not about eliminating pretraining. Second, the native big run was not wasted: the same fleet trained the SONIC tracker checkpoint (14k iterations) that serves as the projection layer for every result in this paper, and the 129k-clip retargeting corpus it was built on supplied the paired data the residual heads trained on. The budget wall stopped the planner, not the program.

(The training-budget-versus-outcome table below carries the per-chain numbers; the native-scratch closed-loop row and exact fleet GPU-hour accounting are marked (*capture*) pending Appendix C.)

Table (real numbers, to finalize): training budget vs closed-loop outcome.

chain	corpus	steps (VQ-VAE/root/pose)	GPU budget	closed-loop (recovery steps)
old deployed (heuristic-era)	—	—	—	6
native scratch	33k clips	~500k/315k/500k	single 8xH100 node	(<i>capture</i>)
native big-run	129k clips	500k/600k/200k	~1,500 H100-h	18–36
source G1	G1-native corpus	2M/2M/2M	~10,000 H100-h	—
training (for reference)			(32xH100, 13 d)	
borrowed core + this paper’s adaptation	33k pairs (heads only)	0 new core steps	~6 RTX-5090-h	{1,2,1}

4 Cloud Training Infrastructure: the 32-GPU Big Run on Nebius

This section documents the cloud fleet behind the native planner attempt of Section 3 and behind the SONIC tracker used throughout the paper. It exists for two reasons: it is the evidence base for the cost-to-2M projection (what a “full schedule” actually demands in practice), and the operational lessons are, in our experience, underdocumented in the literature relative to how much run quality they determine.

Fleet. All cloud training ran on Nebius, whose sponsored GPU credits made the fleet possible: a dedicated tenancy hosting a 4-node cluster, 8x H100 per node, on an InfiniBand fabric. Provisioning was done through the Nebius API — the console’s cluster preset was non-functional at the time — and the API recipe is preserved in Appendix C. Multi-node training ran via torchrun across the fabric. A shared network filesystem held checkpoints and data at rest. (*capture: fleet topology figure.*)

The node-local compute rule. Early runs established a hard operational rule: the shared filesystem is for checkpoints and data at rest *only*. Bulk compute against the share — corpus preprocessing, feature caching, even aggressive log writes — degraded I/O for every node on the fabric. The working pattern is stage-to-local-NVMe, train locally, rsync results back. We state this as a rule rather than an anecdote because we paid for it more than once before it stuck.

Throughput. The planner chain sustained ~217 optimizer steps/min at 32-GPU scale, with the

pose model at 512 samples per GPU for an effective batch of 8192. At that rate the 2M-step source schedule is a weeks-long commitment on this fleet — the concrete content of Section 3’s budget wall. The SONIC tracker trained to its 14k-iteration checkpoint — the shipped, gated model that serves as the projection layer for every closed-loop result in this paper — in a day-class run on the same fleet. (*capture: per-leg wall-clock durations and GPU-hours for the tracker, VQVAE, root, and pose legs; feeds the budget table and Appendix C.*)

Table (to finalize): fleet legs, durations, and outcomes.

leg	hardware	duration	outcome
SONIC tracker (14k)	32 x H100 (4 nodes, IB)	~1-2 days	the shipped/gated tracker used throughout
VQVAE	~16 x H100	(capture)	500k steps, archived; same nodes then moved to pose
root model	16 x H100	(capture)	600k steps
pose model	16 x H100 (after VQVAE)	(capture)	200k steps at wind-down

Operational traps worth publishing. Three cost us real time and would have cost us conclusions had they gone unnoticed:

1. *Rank-0-only checkpointing.* In the multi-node configuration, only rank 0 wrote checkpoints; a naive launch script that assumed per-node checkpoint output silently produced nodes with no resume state. Verify checkpoint provenance before trusting any resume.
2. *Restart transients fake regressions.* Every training restart produces a transient in the sampler-dependent metrics that looks exactly like a data or curriculum regression. The rule we adopted: never interpret a post-restart metric excursion without a control log — a parallel run (or the pre-restart tail) establishing what a clean restart looks like. Several alarming “regressions” in the big run were restart artifacts.
3. *Corpus reshuffle cadence vs metric windows.* The training corpus reshuffles every 250 iterations, which imprints a sawtooth on windowed metrics. Any metric averaged over windows shorter than or incommensurate with the reshuffle period will alias it; we block metrics on 1000-iteration windows. Per-episode error metrics in these logs are additionally measured at termination, which biases them toward failure states; the per-step reward is the only series we found trustworthy for cross-run comparison.

Wind-down. Fleet shutdown followed a fixed order — verify rsync of all run state to the share, copy out the tracker checkpoints, pull required artifacts to local storage, then stop nodes, keeping one node alive for shared-filesystem access until the archive was verified. We mention this because the alternative (stop nodes, then discover a missing artifact) is the single most expensive class of mistake available at wind-down time, and the checkpoint-provenance trap above makes it easy to commit.

The fleet’s planner legs ended at 500k/600k/200k steps (VQVAE/root/pose) as reported in Section 3. The infrastructure did its job; the budget wall was a schedule fact, not an operations failure.

5 System Setting

The planner. Both robots’ planners are the same MotionBricks-family architecture: a multihead VQ-VAE motion tokenizer (4 frames per token), a root model that predicts root trajectory conditioned on a constraint window of 4 start and 4 end frames, and a pose model that fills in body configuration, all operating at 30 fps in a shared skeletal motion representation. A template mode conditions generation on clips from a library — and the library is *data*, loaded alongside the weights, a fact Section 9 exploits. The planner is kinematic: it emits joint-space references and root trajectories, never torques.

The planner contract. The deployed graph takes a 4-frame context of MuJoCo qpos (world frame, WXYZ root quaternion) plus a 4-vector velocity intent (yaw rate, lateral and forward velocity, hip height), and returns a 64-frame plan. Two facts about this contract shaped the whole program. First, the context is *forward-looking* — the daemon supplies the upcoming 4 frames of the currently served plan, not history. Second, the daemon replans at a threshold such that only roughly the first 16 frames of any 64-frame plan are ever served; the plan tail exists but never reaches the robot. Section 6 shows an evaluation harness that ignored the second fact fabricated defects and misled a review round; every visual or quantitative planner evaluation in this paper replicates the daemon’s 16-frame serve cadence.

The target stack. On the target robot, a planning daemon serves plan frames, resampling the planner’s 30 fps output to 50 Hz for the tracker; SONIC, a 50 Hz RL whole-body tracking policy, converts the kinematic reference into torques. All closed-loop results use the shipped, gated tracker checkpoint (14k iterations, Section 4). The tracker is best-effort by construction: it projects the commanded reference onto the target robot’s dynamically feasible manifold rather than following it exactly. This projection property is why a kinematic-planner transfer can work at all, and Section 8 shows it is also why “improving” the reference can backfire.

The morphological preconditions. The transfer’s favorable setting is explicit and, in part, deliberate: the target robot’s skeleton class was built for weight reuse with the source robot’s. Concretely:

- *Identical representation widths.* Both planners use a 34-bone skeleton tree and matched representation widths (414-dim global / 413-dim local); a checkpoint from one initializes the other’s architecture without any shape change. The source spends its last two bone slots on dummy hand-roll bones where the target has real head joints; both pad two dummy toe bones.
- *Naming-level DOF correspondence.* Dropping the target’s two head joints (owned downstream by a head-look overlay, not the planner) leaves an exact 29-to-29 joint correspondence by name. An empirical fit over paired data (Section 6) confirmed every target joint’s best match is its same-named source joint — no hidden permutations, and no sign flips beyond one known wrist-axis naming swap.
- *The only dimensional divergence sits at the qpos boundary:* source qpos is 36 (7 root + 29 DOF), target is 38 (7 root + 31 DOF), and the boundary lives in a weightless converter, not in any checkpoint tensor. The wrap and heads are a semantic remap at this boundary, not a width change.
- *Matched intent semantics.* The velocity-intent vector means the same thing to both planners; only its scale needs mapping.

We report these as preconditions, not incidentals: they are what the method’s cost model depends on, and Section 11 discusses the in-house embodiment (missing waist and wrist joints, sign flips,

roughly half the range of motion) on which we expect the wrap to strain.

Evaluation. The headline closed-loop metric throughout is *recovery steps*: unscheduled corrective steps the tracker takes that the reference did not schedule, counted by a daemon-faithful seam-sweep harness that drives the full planner-daemon-tracker stack in simulation under scripted intents. Recorded operator driving sessions (“tapes”) replayed at original timing against each planner serve as regression artifacts. All results in this paper are simulation; hardware deployment is future work (Section 11).

6 Stage S0: Analytic Alignment

Construction. The wrap is a pair of analytic maps around the frozen core: PhiEncode takes target qpos (38) to source qpos (36) for the context, and PhiDecode inverts the mapping on the 64-frame plan. Its parts:

- *Arms*: per-joint affine maps taken from the robots’ existing retargeting calibration. An independent empirical fit on 60 paired executed clips (37k frames) reproduced the calibration coefficients and showed near-perfect affine structure (R^2 approaching 1.0) — the arms are essentially a change of coordinates.
- *Legs and waist*: fitted affines from the same paired data. Here the affine story is weak — R^2 0.11 to 0.89, hip roll worst — because the target’s legs come from IK against different link lengths, not from a joint map. This measured residual is precisely the budget the learned heads of Section 7 exist to absorb; we knew its size before training anything.
- *Wrist remap*: a vendor naming swap with sign (the source’s wrist roll is the target’s wrist yaw, axis along the forearm) — naming, not geometry.
- *Root*: stride (xy) scaling by 0.8235, an affine on height, and heading passed through. Scaling is anchored at the last context frame so encode and decode agree on the frame of reference.
- *Intent*: the linear velocity channels are scaled *up* by $1/0.8235$ on encode — the decode-side stride scale would otherwise eat commanded speed. This bug was caught by a directional smoke test (commanded 0.4 m/s realized as 0.31) that open-loop reconstruction parity would never have flagged.

Encoder, frozen core, and decoder are fused into a single ONNX graph presenting the target planner’s exact contract, so the daemon, the evaluation harness, and the robot stack run it with zero changes.

Mode routing, and a trap. The source clip library’s inherited mode labels were wrong: kinematic labeling of the clips (pelvis height, speed, yaw rate) showed the modes our lookup table treated as locomotion were all idle variants, so early configurations drove every gait from an idle pose template. The fix is an intent-aware mode router in the graph. The lesson generalizes: never inherit mode semantics from comments; label clip libraries from their kinematics.

Zero-shot results. At walk 0.35 m/s under the seam sweep, the wrap scores {1,2,3,4} recovery steps across four seeds against the deployed native planner’s 6 and the big-run planner’s 18–36. On recorded operator tapes the gap is larger: 15 versus 58 extra steps (tape A) and 8 versus 34 (tape B) against the deployed planner. Replan latency is at parity (p50 44.8–46.0 ms both planners) — the wrap is computationally free. And one capability came gratis: crouch walking at 0.3–0.5 m/s, which neither native planner delivered cleanly, survives with clean rollouts through the wrapped core because the source’s library contains crouch templates.

Table 1 (real numbers): zero-shot closed-loop results.

metric (walk 0.35, sim)	old native	new native	S0 wrap
seam-sweep recovery steps (3–4 seeds)	6	18–36	{1,2,3,4}
operator tape A extra steps	58	—	15
operator tape B extra steps	34	—	8
replan latency p50 (ms)	44.8	—	46.0
crouch walking	no	no	yes (0.3–0.5 m/s)

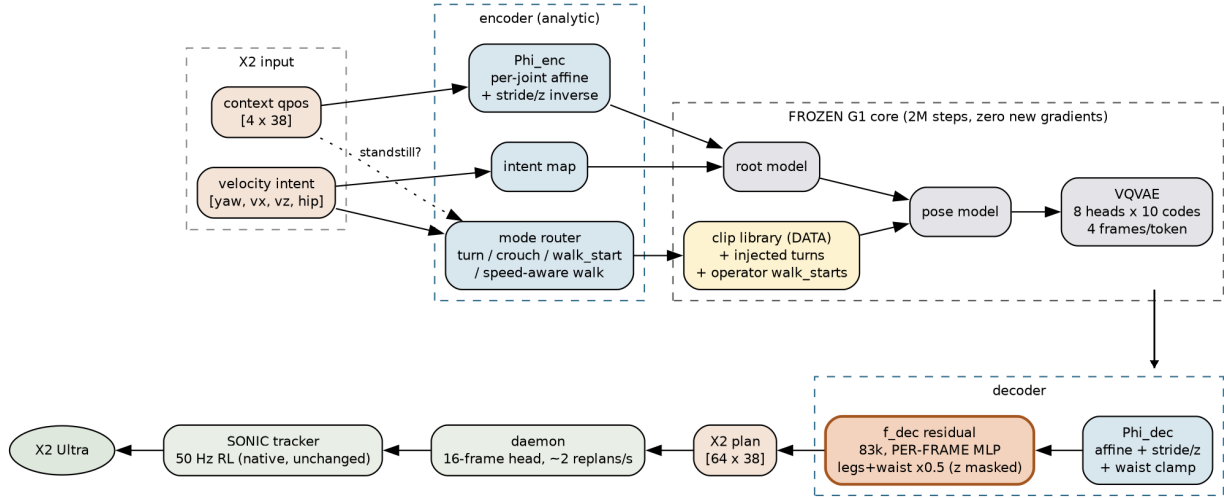


Figure 1: **F1 — System design.** The analytic encoder wrap and intent map feed the frozen G1 core (2M steps, zero new gradients); the mode router selects templates from the clip library, which is *data* — extended with injected turn modes and operator-captured walk-start templates. The decoder applies the analytic inverse plus the 83k-parameter per-frame residual head (legs+waist at half strength, height channel masked), and the plan flows through the unchanged daemon and native SONIC tracker.

Harness fidelity. An early visual review of the wrapped planner reported four defects — splayed turn legs, forward lean, a collapsing run, a crouch that did not crouch. All four were artifacts of a harness that consumed 48 frames per plan (the autoregressive tail) instead of the daemon’s 16-frame serve. Re-rendered at deployment cadence, the defects vanished; the retraction is part of the ledger, and 16-frame serving is now load-bearing in every evaluation tool. Side-by-side with the source robot running the native core, the operator judged the wrapped output to match the source’s motion — the wrap neither adds nor loses style, including the source’s own flaws (its in-place turns slide on both bodies).

Attribution by A/B isolation. High-speed commands fail through the wrap — and the right question is whose failure it is. We ran the same conditions through (A) the wrapped planner with the wrap inside the autoregressive loop, and (B) the native source core decoded one-shot with the same map applied once, both tracked by the same tracker. B failed cell-for-cell like A: the source core’s own speed envelope collapses at high asks natively, and clean fast references fall at the tracker’s coverage boundary on both paths. Speed failures are core envelope plus tracker coverage, not transfer error — and the isolation pattern (in-loop versus one-shot application of the same map) is, we think, the clean instrument for making such attributions.

7 Stage S1: Residual Heads — an Architecture Ladder

Data. The heads train on a paired corpus built from the retargeting pipeline: 33,204 of 33,206 clips cached as frame-exact source/target qpos pairs (alignment verified at lag 0; arm R^2 0.996 median on the walk tier through the analytic affines). Corpus gates matter and we got them wrong twice before getting them right: a variance-normalized R^2 gate cut near-static-arm clips that were actually fine, and a jerk threshold set below the corpus median silently trained the first run on a low-jerk subcorpus. The final gate (arm RMSE, calibrated jerk ceiling, knee-flip drop) keeps ~31k clips. One gate validated itself: a per-clip negative-knee-correlation detector flagged 821 clips, independently rediscovering — from pair statistics alone — a known list of ~808 IK branch-flipped clips from a prior corpus-corruption investigation.

The ladder. Five stages, all zero-initialized residuals over the analytic wrap (bit-exact S0 at step zero), all trained on one workstation GPU:

1. *Linear probe* (~40k params): affine correction terms only. Beat the per-joint analytic fit decisively — cross-frame and cross-joint terms carry real signal. Gate to proceed.
2. *Windowed MLP* (817k): MLP heads on non-overlapping 4-frame windows. Best validation numbers of the program to that point.
3. *Temporal conv* (852k): stationary 1D convolution over frames, built specifically to fix stage 2’s failure mechanism. Matched the MLP’s validation fit.
4. *Per-frame MLP* (83k): no temporal context at all; position-only features; decoder-side only; output masked to legs, waist, and root height. Best validation fit of all five — legs 2.54° , waist 3.02° , height 1.32 mm.
5. *Variable-segment conv* (382k): lower training loss than stage 4, *worse* validation (legs 2.73°). Not gated.

Table 2 (real numbers): offline fit vs closed-loop viability.

heads	params	val legs MAE	val z MAE	recovery steps	falls
none (analytic)	—	5.61°	7.8 mm	{1,2,3}	0
windowed MLP ¹	817k	2.73°	1.4 mm	{21,25,32}	0
temporal conv	852k	2.64°	1.4 mm	—	3/3
per-frame (ours)	83k	2.54°	1.3 mm	{8,16,19}→ see §6	0
variable-seg conv	382k	2.73°	1.4 mm	not gated	—

Failure one: the windowed MLP (offline best, closed-loop disaster). Validation legs 2.73° , height 1.4 mm — and {21,25,32} recovery steps in the seam sweep against S0’s {1,2,3}. No falls; the tracker simply fights the reference continuously. The sweep’s own telemetry located the mechanism: mid-plan acceleration p95 at 3x baseline while seam acceleration rose less — the jitter lives *inside* the plan, not at replan boundaries. Because the heads run on non-overlapping 4-frame windows, a frame’s correction depends on its window slot, so the residual jumps at every window boundary: a

¹Val MAE from the corrected-corpus run; recovery steps from the architecture-identical run 1 (accidental low-jerk subcorpus) — the two runs share the window-slot defect by construction.

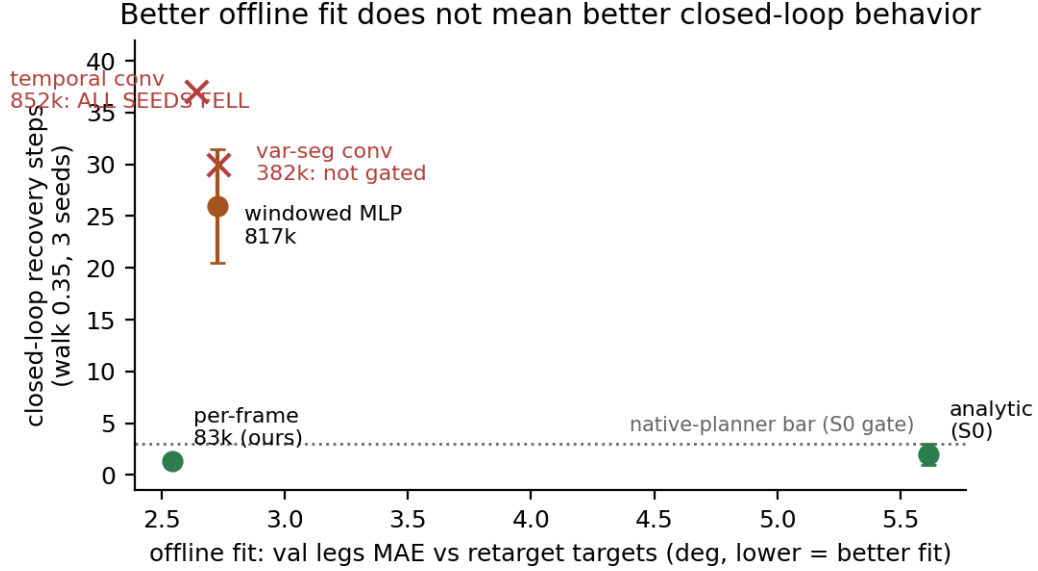


Figure 2: **F2 — Offline fit vs closed-loop viability, per head architecture.** Both large temporal models fit the targets well and fail closed-loop (one destabilized the tracker outright); the 83k per-frame model is simultaneously the best fit and the only architecture at the native bar.

structural 7.5 Hz discontinuity injected into the reference. Per-window validation MAE cannot see it, because every validation window is scored independently — the metric is blind by construction to the failure it needed to catch.

Failure two: the temporal conv (fixed the jitter, fell over). The stationary conv eliminates window slots by design and matched the MLP on validation. In the seam sweep, all three seeds fell — the first falls of the entire program. Telemetry: seam acceleration at 6x baseline, step-to-step jumps of 3.3 rad. The post-mortem found three faces of one failure, train/deploy input mismatch: (a) the encoder head’s receptive field (9 frames) exceeds the deployed context length (4), so at deploy every context frame sits in replicate-padded edge territory the training segments never showed it — and this OOD input corrupts the context that the *frozen generative core* conditions on, the worst possible place to be out of distribution; (b) finite-difference velocity features, trained on mocap-smooth data, amplify core-output frame noise at deploy; (c) the padding effects land exactly at replan handoff frames.

The audit principle. With a frozen core in the loop, the heads’ deploy-time input distribution is defined by the *pipeline*, not the dataset. Every feature and architecture choice must be audited against “what does this see at deploy that training never showed it”: window slots (stage 2), short-context edges (stage 3), velocity noise (stage 3), core output versus mocap (all stages).

The winner is quasi-static. Stage 4 is the audit principle applied as architecture: per-frame (no slots, no edges), position-only (no velocity amplifiers), decoder-only (the core’s inputs stay bit-identical to gated S0, deleting the fall risk class outright). It also has the *best* offline fit at 1/10th the parameters, and stage 5’s capacity-added regression confirms it from the other side: the residual the analytic wrap leaves behind is pose-conditioned geometry, and temporal context buys only failure modes. Training-loss curves were officially useless as a selection signal in this program — stage 5 had the lowest loss on the ladder.

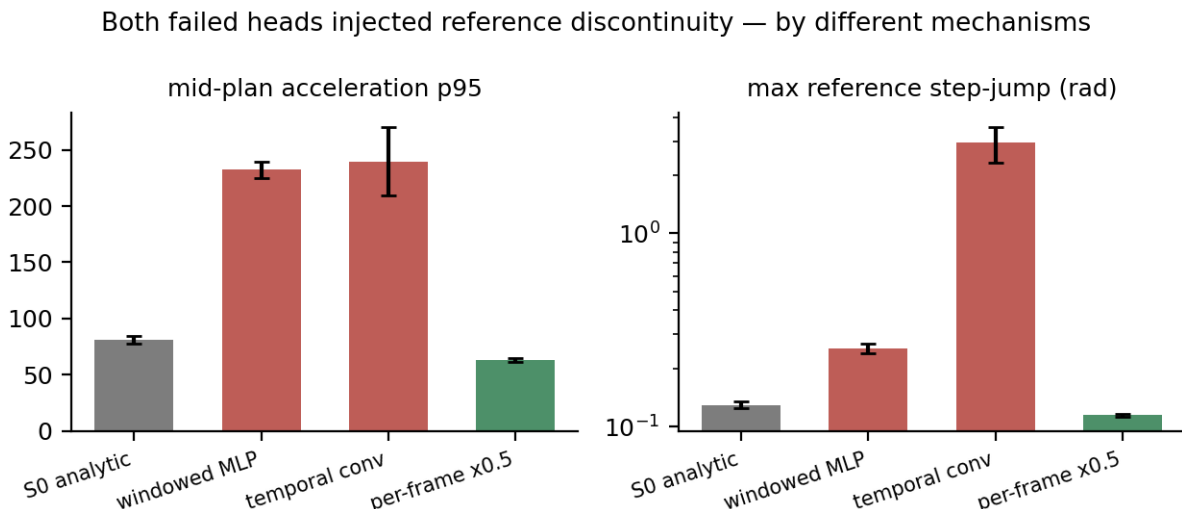


Figure 3: **F3 — The two failure mechanisms in the served reference.** The windowed MLP raises mid-plan acceleration $\sim 3\times$ (window-slot discontinuity); the temporal conv raises the maximum reference step-jump $\sim 25\times$ at replan seams (context-edge corruption upstream of the frozen core). The per-frame heads produce a reference smoother than the analytic baseline.

8 Closed-Loop Selection and the Height Story

Section 7 selected an architecture; this section selects a configuration — and makes the case that the second selection is where the method lives. The instrument is a gate battery run on every candidate: multi-seed seam sweeps (recovery steps, plus reference telemetry: mid-plan acceleration, step-to-step jump), operator-tape replays, an FK stance audit, and an operator feel-test as final acceptance. Offline validation is recorded but never promotes.

The best-fit model fails its first gate. The per-frame heads at full strength scored $\{8,16,19\}$ recovery steps — worse than analytic S0’s $\{1,2,3\}$, despite the best validation table of the program and a reference measurably *smoother* than S0 (mid-plan acceleration p95 53–60 versus 78–84). Continuity was solved; something else was wrong.

The float is load-bearing. The clue was quantitative: $\{8,16,19\}$ matches the $\{11,12,15\}$ we had measured earlier when we “fixed” the analytic wrap’s height convention from the serve side. The wrapped reference floats — FK puts its stance feet above the native planner’s convention — and the heads fit ground-truth height to 1.3 mm, which *removes* the float. We had now run the same ablation from both directions: correct the height analytically (serve-side offset, recovery steps roughly tripled, reverted) and correct it by learning (trained height head, same signature). Offline contact statistics call the float an error; the tracker calls it a convention it depends on. Our working hypotheses — the tracker’s own training references carry an effective float; floated references bias toward earlier foot lift and fewer scuffs — remain untested (Section 11), but the selection consequence is unambiguous either way.

Export-time knobs make one model a family. The heads’ output passes through a channel mask and a scalar strength multiplier that are buffer edits at export — no retraining. Masking the height channel recovered most of the gap ($\{8,6,6\}$): the residual damage was the leg corrections themselves, since ground-truth leg angles encode ground-truth (grounded) foot height, so exact legs

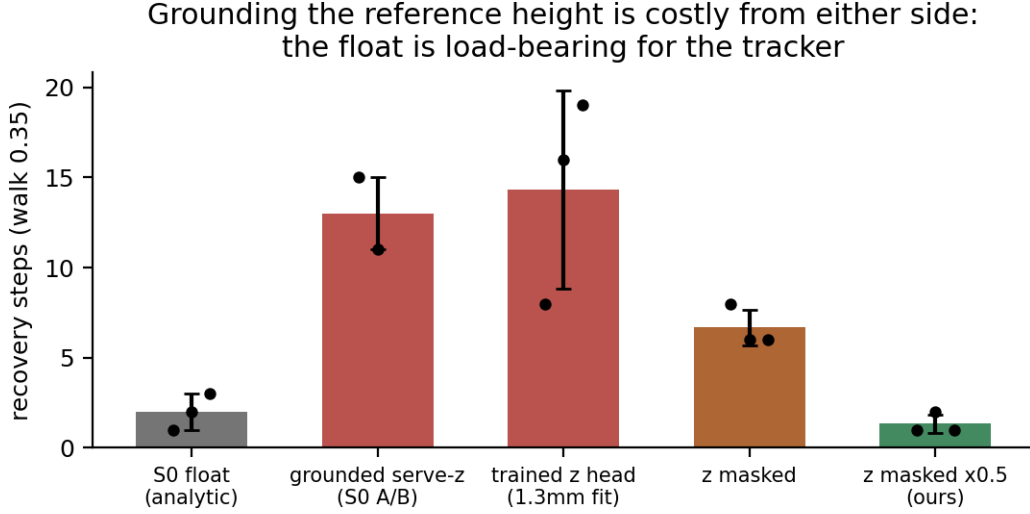


Figure 4: **F4 — The height ablation, two-sided.** Grounding the reference height — whether by offsetting the served root height (S0-era A/B) or by a z head fitting the retarget targets to 1.3 mm — costs several times the recovery steps of keeping the floating convention. Masking the z channel and halving the joint corrections recovers the stability crown.

under a floated root are an internally inconsistent reference. Halving the leg and waist corrections resolved that tension: **height masked, x0.5 strength scores {1,2,1}** — at or below analytic S0’s {1,2,3} — while keeping most of the style gain, with a smoother reference than S0 (acceleration p95 61–65 vs 78–84; step-jump 0.112–0.116 vs 0.124–0.134). One trained model, a searchable configuration family, and the search ran in closed loop at rollout cost only.

Table 3 (real numbers): the winning configuration ladder.

config (per-frame heads)	recovery steps	acc_mid p95	jstep max
full z + legs + waist	{8,16,19}	53–60	0.13–0.16
z masked	{8,6,6}	66–70	0.13–0.15
z masked, ×0.5 strength	{1,2,1}	61–65	0.11–0.12
source wrap (S0)	{1,2,3}	78–84	0.12–0.13

What the winning configuration bought. On the FK audit (method-validated: it reads the old planner at exactly its known 7.68 cm stance convention, band ± 0.2 cm), the shipped configuration walks at 8.87 cm ± 0.4 cm: stance wobble improved from the ± 1.5 cm the operator had flagged by eye to ± 0.4 cm — the program’s target band — *through leg corrections alone*, and the float shrank from +1.8 cm to +1.2 cm with the height channel masked. The remaining +1.2 cm is deliberately left in place (Section 11). On operator tapes, the configuration survives the full 350 s of tape A where S0 fell at 96 s, at roughly half S0’s per-second extra-step rate (26 extra steps over 350 s versus 15 over 96 s), and scores 6 extra steps on tape B against S0’s 8 and the deployed planner’s 34.

The selection ledger. Across the program, offline fit ordered the candidates {windowed MLP,

temporal conv, per-frame} nearly inseparably, and ranked the full-strength per-frame configuration above the half-strength one. Closed loop ordered them {per-frame » windowed MLP » temporal conv} and preferred half strength by a factor of several. Every inversion mattered: promoting by validation would have shipped, in order, a 10x recovery-step regression, a falling robot, and a floating-convention violation. Selection must be closed loop; offline metrics in this program were not merely uninformative but anti-predictive at the decisions that counted.

9 Template Injection and Routing

The frozen core’s template mode conditions generation on clips from a library that ships beside the weights — a documented feature of the source architecture, not a discovery of ours. Our contribution is using it as a *repair channel*: because the library is data, extending it does not violate the frozen-weights constraint, and behaviors the core cannot produce can be grafted in from recordings.

The motivating gap is slow walking — and it is a *source-native* gap, not a transfer artifact. Even on its own robot, the source core authors a surging first stride from standstill; the target robot’s operator, who had previously trained a target-native model on extra-slow walk data specifically because it started more gently, identified the surge immediately and confirmed it on the native reference. Intent-side rate limiting does not fix it (a cold-start ramp A/B at two time constants left the first strides unchanged while making launch feel sluggish): the surge is authored by the generative core from standstill context, reflecting what its training corpus contains. Since the demo requirement was precisely smooth slow walking below 0.6 m/s, we used the template channel to close the gap the corpus left open. This section reports two injections and the capture protocol behind the second, plus the two data-handling traps the pipeline surfaced.

Turn templates. The source core’s in-place turns slide — a source-native style, confirmed on the native reference (Section 6). We encoded the target robot’s proven heuristic-era turn primitives through the wrap into source joint space and appended them as library modes 15/16, routed by a yaw-dominant intent test. The injected step-turns work in direct drive; through the deployed daemon path the slide persists (our suspect is the daemon’s context filtering and 30-to-50 Hz resampling flattening the clips’ subtle weight-shift steps). We report this as an open item: in-place turns remain the accepted, documented inability of the shipped configuration.

Gait-initiation templates, operator-captured. The core surges on its first stride from standstill — confirmed core-native (the source-native column shows it too) and input-conditioned, so the daemon’s velocity ramp cannot remove it, only trade it for sluggishness. The fix inverts the trade: pay the sluggishness *once*. The operator drove a capture session under a deliberately long ramp constant, producing slow, clean standstill-to-walk transitions; we harvested the session by jerk metrics — 15 start attempts, 4 rated GOOD (initiation-to-steady jerk ratio ~1.0, height dip under 10 mm), including one left-lead and one right-lead take — and built them into library modes 17/18. Routing is stateless and in-graph: context xy displacement under 1 cm plus a nonzero forward command routes to walk_start; once the robot is moving, the condition dies naturally. The gate cost is small ({4,3,3} versus {1,2,1}, with the +2 concentrated at the sweep’s stop/start events, where the step counter cannot distinguish a stumble from the recorded weight shift; tape B is neutral at 6=6), and the operator confirmed the result in a drive: “all good... this is real.”

Style follows the template. A related finding closed the loop on gait style: at higher walk speeds the system produced hurried slow-walk strides, because the walk mode rode a slow template

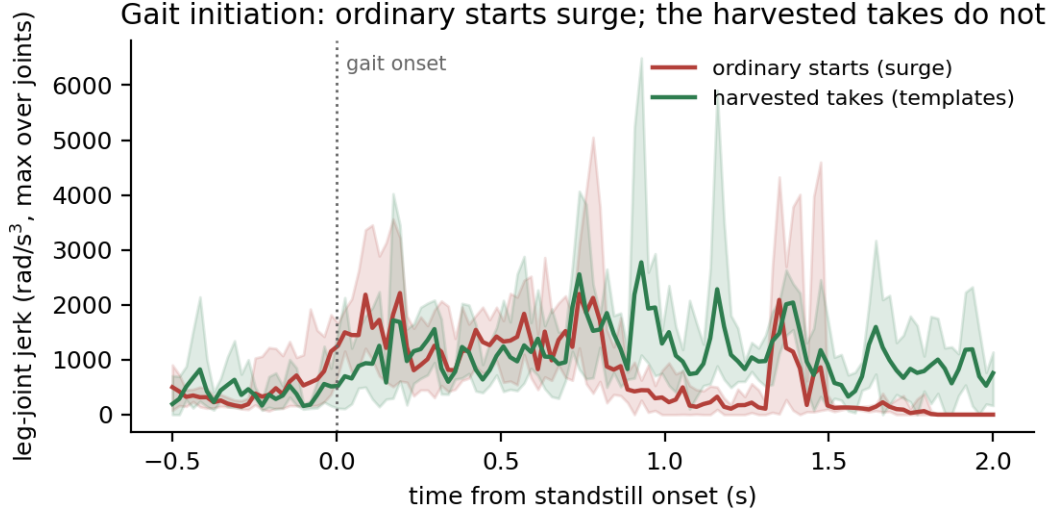


Figure 5: **F5 — Gait initiation in the capture session.** Leg-joint jerk around standstill onset, averaged over the session’s ordinary starts (surging) versus the four harvested takes (initiation jerk at or below their own steady-state level); shaded bands are min–max. The harvested takes became the walk-start library templates.

regardless of commanded speed. A speed-aware template switch (above 0.55 m/s, route to the library’s fast-walk clip) fixed the stride character with, again, zero training. Template selection is a per-intent style knob on a frozen core, and it transfers verbatim across embodiments.

Two tape-parsing traps, reported in full. Both bit during walk_start capture, and the second is the sharpest honesty item in the paper. (1) The capture tape records the *served 50 Hz stream*, not 30 fps planner frames; the first build silently produced 1.67x slow-motion clips. Caught on review; rebuilt with explicit rate detection and resampling. (2) The tape’s per-frame record stores its root quaternion in XYZW order; the planner convention is WXYZ. Reading one as the other scrambles root orientation while leaving joint columns coincidentally aligned — and the first walk_start build shipped wrong-orientation clips that *passed the quantitative gate and an operator feel-test*. Only the FK stance audit, which reads absolute foot heights and returned an absurd number, caught it. The library was rebuilt from the archived session with the correct parse, re-gated, and re-confirmed. Two lessons: every tape consumer now goes through one shared parser; and gates that measure relative stability can certify a reference that is *wrong* in ways the tracker absorbs — absolute-geometry audits belong in the battery.

Limits of the library. Injection cannot exceed the core’s data floor. Commanded crouch depth saturates near the source corpus’s lowest hip height regardless of the command, and the crouch route currently rides a crouch-run template whose flight phases show up as 29% airborne reference frames — a calmer crouch-walk template is queued. The library is a control surface over behaviors the core can already express, not a mechanism for synthesizing new ones.

10 Results

Table 4 carries the headline numbers; this section reads them.

Table 4 (real numbers): final system vs baselines.

	old native	S0 wrap	final (ours)
seam recovery steps	6	{1,2,3}	{1,2,1}
tape A (350 s, 84 stops)	58 extra	fell @96 s	full tape, 26 extra
tape B (103 s, 16 stops)	34 extra	8 extra	6 extra
stance wobble (p10–p90)	± 0.2 cm	± 1.5 cm	± 0.4 cm
stance float vs native	0	+1.8 cm	+1.2 cm
reference jitter (acc_mid p95)	—	78–84	61–65
operator verdict	—	“impressive zero-shot”	“ready to ship”

Stability. The final configuration (per-frame heads, height masked, half strength, plus walk-start templates) holds the program’s recovery-step crown: {1,2,1} on the multi-seed seam sweep against the deployed native planner’s 6 and analytic S0’s {1,2,3}; the template-extended build gates at {4,3,3} with the difference concentrated at stop/start events where the counter reads the recorded weight shift as a step. On operator driving tapes — the closest instrument we have to deployment — the system survives the full 350 s of tape A where analytic S0 fell at 96 s, with roughly half S0’s per-second extra-step rate (26 over 350 s versus 15 over 96 s), and scores 6 extra steps on tape B against S0’s 8 and the deployed planner’s 34. Relative to the deployed baseline the operator drives today, that is roughly an order of magnitude fewer corrections on tape B and a fall converted into a full-tape survival on tape A.

Operator driving tapes: replayed intent, identical protocol

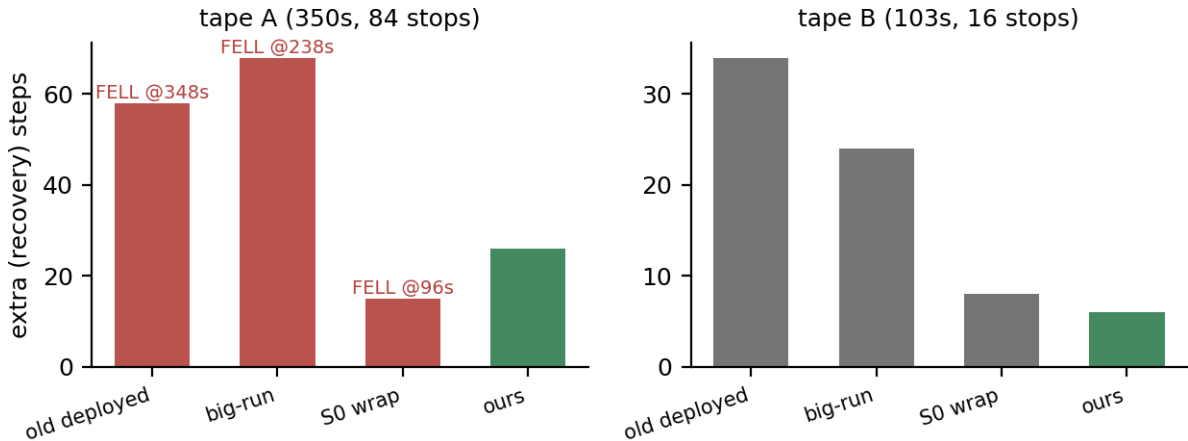


Figure 6: **F6 — Operator driving tapes, replayed under identical protocol.** On tape A the S0 wrap falls at 96 s; the final configuration survives the full 350 s with roughly half the per-second correction rate, and improves on tape B as well.

Reference quality. The final reference is smoother than the analytic wrap’s (mid-plan acceleration p95 61–65 versus 78–84; step-jump 0.112–0.116 versus 0.124–0.134) and better grounded: stance

wobble ± 0.4 cm against the wrap’s ± 1.5 cm (native convention: ± 0.2), float reduced from $+1.8$ cm to $+1.2$ cm — through leg corrections alone, with the height channel masked (Section 8). Stop poses moved as a side effect: knee angle at cut went from the wrap’s $22\text{--}42^\circ$ to $21\text{--}35^\circ$ in the shipped configuration, toward the native planner’s stop style ($\sim 16^\circ$) — which may absorb part of a queued daemon stop-logic retune for free.

Cost. Replan latency is at parity with the native planners (p50 44.8–46.0 ms) — the wrap and heads are computationally free at inference — and the entire adaptation consumed ~ 6 workstation-GPU hours against the $\sim 10,000$ H100-hours the source schedule represents (Section 3).

Capabilities. The capability matrix is deliberately honest. Slow walk at 0.3 m/s — the demo requirement that started the program — is operator-verdict ready to ship, with one scoped caveat: the residual foot taps and recovery steps that remain concentrate almost entirely in tight moving turns, not in straight-line gait (the operator’s estimate from the final drive was smooth “97% of the time”, with the exceptions in turns). We treat tight-turn behavior as the top open item, distinct from the in-place slide below. Crouch walking at 0.3–0.5 m/s is a new capability neither native planner had, inherited from the source library, with a known depth floor. In-place turns slide (accepted, documented inability; the injected step-turn templates work only in direct drive). High-speed gaits are out of envelope by the A/B isolation of Section 6 — a bound belonging to the source core’s corpus and the tracker’s coverage, not to the transfer.

Operator verdicts function as acceptance data in this program, and we quote the load-bearing ones: “pretty impressive for a zero-shot port” (S0 first drive); “walk is much better” (winning heads configuration); “all good. . . this is real” (walk-start build, after the orientation fix); and, on the shipping configuration, that the 0.3 m/s demo walk “looks ready to ship.”

The thesis, third instance. The results table is also the selection ledger’s summary row. Every column where the final system wins was selected *against* an offline metric’s advice at least once: the planner itself (offline-better native chain lost in closed loop, Section 3), the head architecture (best-validation models scored 10x worse or fell, Section 7), the configuration (exact height fitting tripled recovery steps; half-strength beat full, Section 8). Offline fit answered “does the head reproduce the retarget targets”; closed-loop rollout answered “can the tracker stand on the result.” The paper’s method is, finally, just the discipline of never confusing the two questions — the numbers above are what that discipline returned.

10.1 Qualitative comparison: source and target, same commands

Kinematic reference playback, source robot (left columns, native G1 core) vs target robot (right columns, wrapped planner with heads), identical velocity commands; two gait phases per cell.

11 Limitations and Open Fronts

One embodiment pair, and a favorable one. Every result in this paper is one source-target pair, and the target’s skeleton was deliberately built for weight reuse with the source’s: identical 34-bone tree, matched representation widths, joint-for-joint naming correspondence (Section 5). We characterized the preconditions — calibration-exact arm affines, leg residuals absorbable by 83k parameters, matched intent semantics — but we have not tested where they break. An in-house stress pair exists (an embodiment missing waist and wrist joints, with sign flips and roughly half the source’s range of motion) on which we expect the wrap to visibly strain; that experiment, which



Figure 7: **F9a — Slow walk, 0.3 m/s.** The demo-critical gait: the target reproduces the source's stride character through the wrap; this is the configuration the operator signed off as demo-ready.



Figure 8: **F9b — Run, 3.0 m/s command.** Out-of-envelope stress case (the tracker's coverage, not a transfer defect — §5); stride style follows the fast-walk template on both sides.

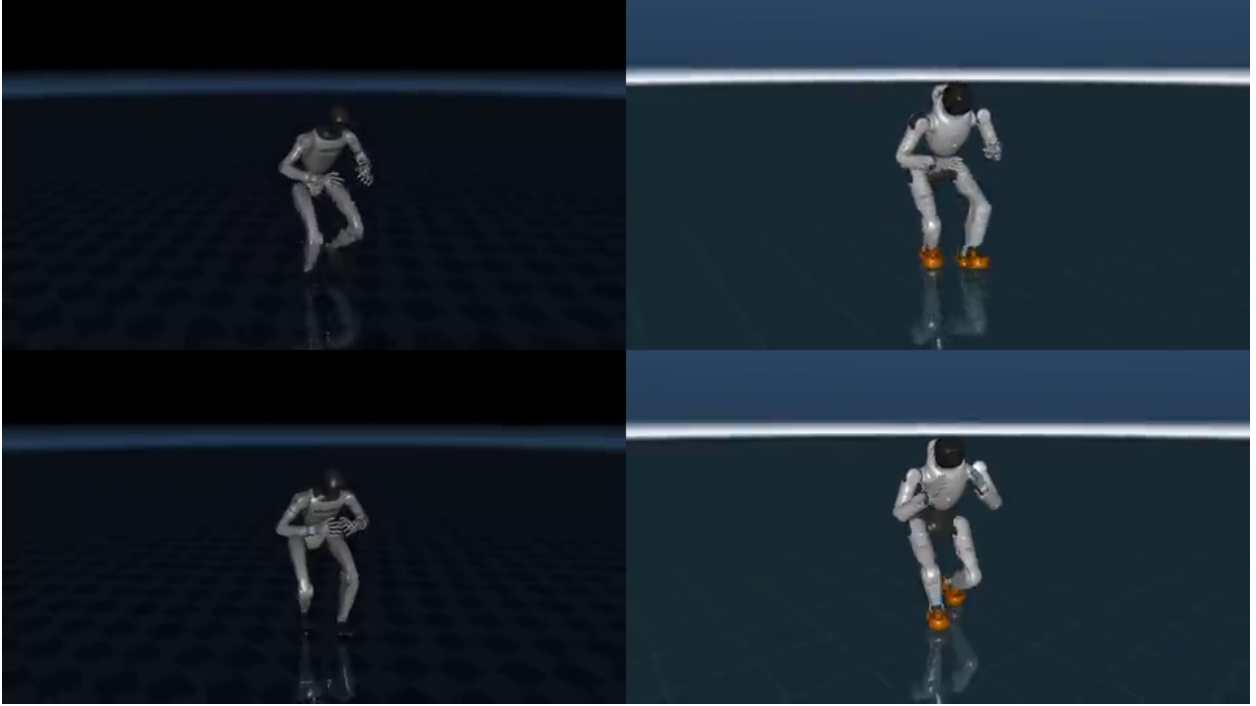


Figure 9: **F9c — Crouch walk (hip 0.55 m).** A capability neither native X2 planner had; inherited from the source core’s crouch template for free. Depth saturates at the source corpus’s floor (§9).

would map the method’s boundary, has not been run. Until it is, the claim is “sibling embodiments,” not more.

Simulation only. All closed-loop results are simulation with the shipped tracker checkpoint. The stack is deploy-shaped — the fused graph presents the deployed planner contract, the harness drives the real daemon, and latency is at parity — but hardware verification is future work, and the program’s own history (Section 9’s feel-test that passed a wrong-orientation build) counsels humility about any gate short of the robot.

One tracker. Closed-loop viability is defined here against a single tracker checkpoint. The float ablation shows the reference conventions that matter are *tracker-specific* — a retrained tracker could move the basin, invalidating the selected configuration (though not the selection protocol). We did not test configuration robustness across tracker versions beyond one incidental replication.

Known behavioral gaps, with owners. In-place turns slide; the injected step-turn templates work in direct drive but not through the daemon path (Section 9) — the accepted, documented inability of the shipped configuration. Residual taps and recovery steps now concentrate in tight turns, the operator’s remaining complaint. Commanded crouch depth saturates at the source corpus’s floor, and the crouch template’s flight phases (29% airborne reference frames) await a calmer clip. First-stride initiation is improved by the walk-start templates but not polished. High-speed gait is out of envelope — bounded by the source core’s corpus and the tracker’s coverage, not by the transfer — and we treat it as a stress canary, not a target.

The remaining float. The shipped reference still floats +1.2 cm above the native stance convention. We know how to remove it (the height head fits to 1.3 mm) and we know removing it costs ~3x

recovery steps. The principled fix — training the height target against tracker trackability rather than geometric ground truth, so the heads learn the convention the tracker actually prefers — is designed (trackability-weighted regression, with SONIC rollout scores as per-chunk weights) but unused. Likewise unused: closed-loop supervised training of the heads (CAT-K-style rollouts), which we expected to need and did not.

Mechanism gaps. The float’s load-bearing-ness is established by ablation from both directions, but the mechanism is hypothesis-level (tracker training references carrying their own effective float; early-lift bias). The daemon-path turn-slide suspect (context filtering plus resampling flattening weight-shift steps) is likewise untested. And the recovery-step counter itself has known blind spots: it reads recorded weight shifts as extra steps at stop/start events and cannot see sub-threshold foot taps — the seam sweep’s documented run-to-run variance means small differences between passing configurations ($\{1,2,1\}$ versus $\{1,2,3\}$) should be read as parity, not ranking.

Selection cost. Closed-loop selection is the paper’s method, and its price should be stated: every candidate costs rollouts (multi-seed sweeps plus tape replays), and the export-knob family search multiplies candidates. It was cheap here because the heads are tiny and simulation is fast; for a setting where rollouts are expensive, the anti-predictiveness of offline metrics we report is a genuinely inconvenient result.

12 Conclusion

We set out to buy a smooth slow walk for a demo and ended up with a result about model reuse. A generative motion planner trained to 2M steps on one humanoid drives a morphologically close humanoid at deployment quality with zero gradient steps on its weights: an analytic per-joint wrap achieves zero-shot transfer that beats both planners natively trained for the target robot, and 83k parameters of per-frame residual heads — selected in closed loop, tempered by export-time masking and strength scaling — close most of the remaining gap. The adaptation cost ~6 workstation-GPU hours against the ~10,000 H100-hours the source training schedule represents. For sibling embodiments — and we have characterized what “sibling” required here — frozen-planner reuse is practical today.

The transferable artifact, though, is the evaluation discipline, not the wrap. Three times in this program, offline metrics and closed-loop viability disagreed, and three times the offline metric was not just uninformative but anti-predictive: the natively trained planner that improved every offline number and lost in closed loop; the best-validation head architectures that destabilized or felled the tracker through deploy-time input distributions their training never showed them; the millimeter-exact height fit that removed a floating convention the tracker depends on. The working principles we extracted — select by rollout, never by reconstruction; audit every head input against what the pipeline (not the dataset) will feed it; treat the tracker as a projection operator and optimize basin margin, not exactness; keep absolute-geometry audits in the battery because stability gates can certify wrong references — are, we believe, worth more to a reader than the specific affine tables in Appendix A.

We reported the ledger in full: two closed-loop-falsified architectures (and a third rejected on validation) with their mechanisms, two reverted “fixes,” two tape-parsing traps including one that passed a gate and a human feel-test, and a mode table that was wrong in a way that silently degraded every early result. The next experiments are named: the mismatched in-house embodiment where the wrap should strain, trackability-weighted height training for the last 1.2 cm of float, and the

robot itself.

13 Acknowledgements

We thank **Nebius** for sponsoring the GPU credits that funded the cloud training fleet described in Section 4 — the SONIC tracker and both native planner-chain attempts were trained entirely on that infrastructure. We thank the MotionBricks authors for releasing the pretrained G1 planner checkpoints that made this study possible.

14 Authorship Note

This work was carried out as a human–AI collaboration. The first author set direction, constraints, and acceptance criteria, performed all robot and simulator evaluation driving, recorded the gait-initiation takes, and made every promotion decision. The AI co-author (Claude Fable 5, Anthropic, operating in Claude Code) implemented the wrap, training, and evaluation code, ran and analyzed the experiments under the first author’s direction, maintained the experimental ledger, and drafted this manuscript, which the human authors reviewed. The human authors bear responsibility for the manuscript’s claims. Venues that do not permit AI systems as authors should treat this note as the required disclosure, with the AI contribution moved to the acknowledgements.

15 Appendix A: Joint Alignment Tables

[Full per-joint affine table with fit statistics.]

16 Appendix B: Evaluation Harness

[Seam-sweep specification; the 16-frame daemon-faithful serving lesson; tape formats and parsing traps.]

17 Appendix C: Reproducibility

[Commits, seeds, run IDs, corpus manifest, exact commands.]

References

- Aberman, Kfir, Peizhuo Li, Dani Lischinski, Olga Sorkine-Hornung, Daniel Cohen-Or, and Baoquan Chen. 2020. “Skeleton-Aware Networks for Deep Motion Retargeting.” *ACM Transactions on Graphics* 39 (4). <https://doi.org/10.1145/3386569.3392462>.
- Araújo, João Pedro, Yanjie Ze, Pei Xu, Jiajun Wu, and C. Karen Liu. 2025. “Retargeting Matters: General Motion Retargeting for Humanoid Motion Tracking.” *arXiv Preprint arXiv:2510.02252*.
- Harithas, Sudarshan, and Srinath Sridhar. 2025. “MotionGlot: A Multi-Embodied Motion Generation Model.” In *IEEE International Conference on Robotics and Automation (ICRA)*.
- He, Tairan, Zhengyi Luo, Wenli Xiao, Chong Zhang, Kris Kitani, Changliu Liu, and Guanya Shi. 2024. “Learning Human-to-Humanoid Real-Time Whole-Body Teleoperation.” In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. “LoRA: Low-Rank Adaptation of Large Language Models.” In *International Conference on Learning Representations (ICLR)*.
- Jin, Ming, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, et al. 2024. “Time-LLM: Time Series Forecasting by Reprogramming Large Language Models.” In *International Conference on Learning Representations (ICLR)*.
- Kumar, Ananya, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. 2022. “Fine-Tuning Can Distort Pretrained Features and Underperform Out-of-Distribution.” In *International Conference on Learning Representations (ICLR)*.
- Lee, Sunmin, Taeho Kang, Jungnam Park, Jehee Lee, and Jungdam Won. 2023. “SAME: Skeleton-Agnostic Motion Embedding for Character Animation.” In *SIGGRAPH Asia 2023 Conference Papers*. <https://doi.org/10.1145/3610548.3618206>.
- Luo, Zhengyi, Jinkun Cao, Josh Merel, Alexander Winkler, Jing Huang, Kris Kitani, and Weipeng Xu. 2024. “Universal Humanoid Motion Representations for Physics-Based Control.” In *International Conference on Learning Representations (ICLR)*.
- Silver, Tom, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. 2018. “Residual Policy Learning.” *arXiv Preprint arXiv:1812.06298*.
- Villegas, Ruben, Jimei Yang, Duygu Ceylan, and Honglak Lee. 2018. “Neural Kinematic Networks for Unsupervised Motion Retargetting.” In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 8639–48.
- Wang, Tingwu, Olivier Dionne, Michael de Ruyter, David Minor, Davis Rempe, Kaifeng Zhao, Mathis Petrovich, et al. 2026. “MotionBricks: Scalable Real-Time Motions with Modular Latent Generative Model and Smart Primitives.” *ACM Transactions on Graphics* 45 (4). <https://doi.org/10.1145/3811334>.
- Yang, Ming, Tao Yu, Feng Li, and Hua Chen. 2026. “Any2Any: Efficient Cross-Embodiment Transfer for Humanoid Whole-Body Tracking.” *arXiv Preprint arXiv:2605.23733*.
- Zhang, Jiaxu, Junwu Weng, Di Kang, Fang Zhao, Shaoli Huang, Xuefei Zhe, Linchao Bao, Ying Shan, Jue Wang, and Zhigang Tu. 2023. “Skinned Motion Retargeting with Residual Perception of Motion Semantics & Geometry.” In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Zhang, Lvmin, Anyi Rao, and Maneesh Agrawala. 2023. “Adding Conditional Control to Text-to-Image Diffusion Models.” In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Zhang, Zhejun, Peter Karkus, Maximilian Igl, Wenhao Ding, Yuxiao Chen, Boris Ivanovic, and Marco Pavone. 2025. “Closed-Loop Supervised Fine-Tuning of Tokenized Traffic Models.” In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.